

StartupFund AI Indian Startup Funding Amount Predictor

"Turning investor characteristics into meaningful funding estimates"

PRESENTED BY

Prem Thakare

COURSE

Feature Engineering & MLOps

PROJECT TYPE

Individual Mini Project

DOMAIN

Indian Startup Ecosystem

Raw investor data carries no business meaning — we must engineer it

- Investor type alone says nothing about funding capacity or behavior
- founded_year is a calendar fact, not a measure of maturity or experience
- Portfolio count $\neq$ portfolio effectiveness — exits per investment reveals quality
- Core challenge: transform raw columns into signals an ML model can actually use

SO WHAT

The gap between raw data and business meaning is exactly where feature engineering creates value.

StartupFund AI delivers end-to-end ML from raw investor data to INR prediction

- Full pipeline: Investor Data → Preprocessing → Feature Engineering → Feature Selection → Regression → Funding Amount (INR)
- Downstream delivery via FastAPI REST endpoint and Streamlit interactive UI
- Fully reproducible experiment tracking with DVC + MLflow
- Containerized with Docker, automated with GitHub Actions CI/CD

SO WHAT

Every stage — from raw data ingestion to user-facing prediction — is part of a single reproducible system.

The model cannot understand business meaning — feature engineering bridges the gap

Raw Column	Engineered Feature	Transformation	Business Signal
founded_year	investor_age_years	current_year – founded_year	Years of investor maturity
portfolio + exits	exit_success_rate	exits ÷ portfolio	Historical exit effectiveness
portfolio info	portfolio_density	Activity per time unit	Portfolio activity intensity

SO WHAT

Each transformation encodes domain knowledge that raw numbers cannot express on their own.

Six feature groups convert raw investor columns into actionable ML signals

Group	Raw Input	Engineered Features	Domain Purpose
Investor Age	founded_year	investor_age_years	Proxy for experience and maturity
Exit Behavior	portfolio + exits	exit_success_rate, exit_intensity	Quality of historical investments
Portfolio Activity	portfolio data	portfolio_velocity, portfolio_density	Investment pace and breadth
Investment Capacity	multiple fields	investment_capacity	Estimated capital deployment signal
Cyclical Time	month	month_sin, month_cos	Seasonality without ordinal bias

Six sequential phases transform raw data into a model-ready, governed feature set

Phase	Name	Key Activities
1	Foundations	Numerical / categorical / binary / ordinal classification; train/test separation
2	Cleaning & Preparation	Missing value imputation, indicators, scaling, encoding, transformations
3	Feature Creation	Investor age, exit success, portfolio density, capacity, cyclical, interactions
4	Feature Selection	Variance filtering, correlation, statistical tests, model-based, RFE
5	Dimensionality Reduction	PCA, variance preservation, feature grouping for redundancy removal

Four preprocessing steps make raw investor data safe and compatible for modeling

Step	Purpose	Output
Missing Value Handling	Impute or flag absent data without introducing bias	Complete feature matrix
Encoding	Convert categorical variables to numerical representations	Numeric-only input
Scaling	Normalize feature ranges for algorithm compatibility	Unit-scaled features

SO WHAT

Preprocessing is applied only on training data — test data is transformed using fitted transformers, never re-fitted.

Leakage-free pipeline design is the most critical methodological decision in this project

✗ BAD PIPELINE — LEAKAGE PRESENT

- Preprocessing before Train/Test split
- Scaler/encoder learns from train + test
- Target-derived features included
- Future/post-funding information included
- Test-set information influences model training
- Evaluation becomes optimistically biased

✓ GOOD PIPELINE — LEAKAGE PREVENTED

- Raw data → remove leakage-prone columns
- 80/20 Train/Test split
- Feature engineering + preprocessing inside Pipeline
- Scalers/encoders fitted only on training data
- Test data only passes through transform()
- Future/post-funding information excluded
- Target-derived min/max investment features excluded
- Final metrics calculated on untouched test data

SO WHAT

"Evaluation is meaningful only when test set information is kept isolated." — Pipeline + ColumnTransformer enforce this.

Seven selection techniques remove noise, redundancy, and overfitting risk from the feature set

Technique	Type	Role in Pipeline
Variance Threshold	Filter	Remove near-zero variance features
Correlation Analysis	Filter	Eliminate linearly redundant features
RFE (Recursive Feature Elimination)	Wrapper	Iteratively remove weakest-ranked features
Model-based Importance	Embedded	Use tree or linear model coefficients
Lasso / ElasticNet	Embedded	L1 regularization drives coefficients to zero
Permutation Importance	Model-agnostic	Measure impact by shuffling each feature

PCA compresses correlated engineered features into fewer components without losing signal

- Target: [INSERT ACTUAL VARIANCE RETENTION %] of total variance retained after reduction
- Components reduced from [INSERT ORIGINAL FEATURE COUNT] to [INSERT REDUCED COMPONENT COUNT]
- Applied after feature engineering — operates on already-meaningful signals, not raw columns
- Trade-off: components are linear combinations of originals, reducing direct interpretability

SO WHAT

"PCA is not automatically better — useful only when dimensionality and redundancy justify the interpretability cost."

Ridge Regression baseline provides a disciplined reference point for evaluating feature impact

- Target variable: expected funding amount in INR — continuous supervised regression task
- Baseline model: Ridge Regression — regularized linear model, interpretable, fast to train
- Final model: [INSERT ACTUAL FINAL MODEL NAME] — selected after experiment comparison in MLflow
- Evaluation metrics: R^2 (explained variance), MAE (average error magnitude), RMSE (penalizes large errors)

SO WHAT

"A baseline gives a reference point against which feature engineering and model changes can be measured." — without it, improvement is unverifiable.

Honest performance comparison across baseline, engineered, and final model configurations

Configuration	R ²	MAE (INR)	RMSE (INR)
Baseline — Ridge, raw features	[INSERT VALUE]	[INSERT VALUE]	[INSERT VALUE]
+ Engineered Features added	[INSERT VALUE]	[INSERT VALUE]	[INSERT VALUE]

SO WHAT

"The experiment demonstrates that feature engineering alone cannot overcome limitations in the underlying dataset. The value lies in the methodology, not just the metric."

Five visualizations reveal dataset structure, feature relationships, and model signals

- Funding Amount Distribution — histogram showing target variable spread and skew pattern
- Portfolio Activity vs Funding — scatter plot revealing investment scale correlation
- Exit Success Rate vs Funding — scatter showing exit quality as a funding predictor signal
- Feature Importance — horizontal bar ranking top engineered features by model weight

Six MLOps tools form a production-grade infrastructure from data versioning to user interface

Tool	Role	What It Solves
DVC	Data & pipeline versioning	Track dataset and code changes together reproducibly
MLflow	Experiment tracking & registry	Log metrics, params, artifacts; compare runs; register models
Docker	Runtime containerization	Identical environment on every machine — eliminates "works on my machine"
GitHub Actions	CI/CD automation	Automate testing, Docker build, and publish on every commit
FastAPI	REST prediction API	Serve model predictions as a low-latency HTTP endpoint

Training pipeline produces the model; inference pipeline serves it — both share feature logic

TRAINING PIPELINE

- DATA → DVC Pipeline → Training Script
- MLflow logs all metrics, params, and artifacts
- Best model registered in MLflow Model Registry
- Docker image built and pushed via GitHub Actions

INFERENCE PIPELINE

- USER → Streamlit UI → FastAPI API Request
- FastAPI applies same preprocessing + feature engineering
- Trained model loaded from registry generates prediction
- Funding estimate (INR) returned to Streamlit → USER

SO WHAT

Training and serving share identical feature engineering logic — ensuring no training-serving skew in production.

StartupFund AI is a fully deployed product — not a notebook, a live application

- Step 1: User enters investor info — type, sector, stage, location, portfolio size, exits, founded year
- Step 2: Streamlit submits a POST request to the FastAPI prediction endpoint
- Step 3: Feature engineering pipeline processes the input in real time on the server
- Step 4: Trained regression model generates the expected funding estimate in INR

SO WHAT

Live Application: [INSERT ACTUAL APPLICATION URL] — demo the running product during the presentation.

Six pillars define what makes StartupFund AI an MLOps project, not just an ML experiment

Pillar	Tool / Practice	Function
DATA	DVC	Version the data and pipeline — track every dataset change
FEATURES	Feature Engineering	Convert raw columns into business signals with domain knowledge
EXPERIMENTS	MLflow	Compare experiments reproducibly — every run logged and queryable
DEPLOYMENT	Docker	Package the complete runtime — same image in dev, CI, and production
AUTOMATION	GitHub Actions	Automate build, test, and publish on every code push

Honest assessment: a sophisticated pipeline cannot compensate for limited underlying signal

LIMITATIONS IDENTIFIED

- Dataset size constrains model generalization capacity
- Synthetic or mock data may not reflect real funding distributions
- Target variable (funding amount) contains noise and extreme outliers
- Missing real-world signals: team quality, traction, market conditions

KEY LEARNINGS

- Feature engineering cannot create information absent from the data
- Leakage-free pipelines are non-negotiable — not optional best practice
- MLflow experiment tracking reveals what actually improves vs. what feels like progress
- DATA QUALITY + FEATURE QUALITY + MODEL QUALITY + MLOPS = RELIABLE SYSTEM

SO WHAT

"A sophisticated pipeline does not guarantee strong predictions if underlying data has limited signal."

StartupFund AI demonstrates the complete ML lifecycle from raw data to deployed product

- Feature engineering converts raw investor columns into meaningful business signals — bridging data and domain
- Leakage prevention and reproducible pipelines make evaluation trustworthy — without isolation, metrics are meaningless
- MLOps (DVC + MLflow + Docker + GitHub Actions + FastAPI + Streamlit) turns a trained model into a maintainable product
- Full journey: RAW DATA → CLEANING → ENGINEERING → SELECTION → REGRESSION → TRACKING → DEPLOYMENT → APPLICATION

SO WHAT

The primary contribution is methodological: a reproducible, leakage-free, end-to-end ML system — independent of final R^2 .

THANK YOU & Q&A

FEATURE ENGINEERING & MLOPS — INDIVIDUAL MINI PROJECT

StartupFund AI Indian Startup Funding Amount Predictor

"Turning investor characteristics into meaningful funding estimates"

AUTHOR

Prem Thakare

GITHUB

[https://github.com/PremThakareitm/
Mini-Project-Assignment](https://github.com/PremThakareitm/Mini-Project-Assignment)

LIVE APP

[https://mini-project-assignment-
d3wdydm6xolqqzwnubth8p.streaml
it.app/](https://mini-project-assignment-d3wdydm6xolqqzwnubth8p.streamlit.app/)

STACK

scikit-learn · MLflow · DVC ·
Docker · FastAPI · Streamlit